

MODULE3

PythonBDDRestful

Name	Ayush Kar
Program/Year	CSIT /4th Year
Institution	InstituteofEngineering& Management,Kolkata
ModuleCovered	Module 3–PythonBDDRestful
Experiments	Experiments3.1–3.5
Date	19 TH September,2026

Index— Module3Experiments

Exp.No.	Experiment Title	Status
3.1	PythonBasicsforRESTAPIAutomation	Completed
3.2	PythonHTTP Libraries–GET,POST, PATCHandDELETE	Completed
3.3	APIAutomation–Configuration,Sessions andReusableFunctions	Completed
3.4	HandlingHTTPResponses–StatusCodes, HeadersandExceptions	Completed
3.5	BDDwithPythonandBehave–RESTAPI Scenario	Completed

Experiment3.1 :PythonBasics

1. ProblemStatement

REST APIautomation needs basic Python concepts such as variables, dictionaries, functions and JSON data. The task is to use these concepts to read information from a REST API.

2. Objective

To understand the Python basics required for REST APIautomation and access values returned in JSON format.

3. Tools,Software&ConceptsUsed

- Python3.x
- PyCharm
- Requestslibrary
- JSON/dictionarydata
- RESTAPI

Concepts:variables,dictionaries,functions,JSONdata,importingmodulesandbasicoutput

4. SourceCode/ImplementationSteps

```
importrequests

url =

"https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url)
data= response.json()

print("Status Code:",response.status_code)
print("Post ID:", data["id"])
```

5. Output

5.1 TERMINALOUTPUT



```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\Mod3\exp3.1.py
Status Code: 200
Post ID: 1
Title: sunt aut facere repellat provident eccaecati excepturi optio reprehenderit
User ID: 1
Process finished with exit code 0
```

Fig3.1.1:Terminaloutputshowingtheexecutionresult.

5.2 API RESPONSE

The REST API returned the requested posts successfully with HTTP status code 200.

6. Result, Observation & Conclusion

Result:

The Python program was executed successfully, and a GET request was sent to the REST API. The API returned a successful response with status code **200**.

Observation:

The response received from the API was converted from JSON format into Python dictionary data. The required values such as Post ID, Title, and User ID were accessed using dictionary keys and displayed in the terminal.

Conclusion:

This experiment demonstrated how basic Python can be used for REST API automation. The terminal output confirmed that the API data was successfully received, processed, and displayed.

Experiment3.2 :PythonHTTPLibraries

1. ProblemStatement

Python HTTP libraries are used to send requests to RESTAPIs. The task is to practise GET, POST, PATCH and DELETE requests and observe their response status codes.

2. Objective

To understand howthe Requests library sends different HTTP methods and howpayloads are passed to an API.

3. Tools,Software&ConceptsUsed

- Python3.x
- PyCharm
- Requestslibrary
- JSON
- RESTAPI

Concepts:GET,POSTwithJSONpayload, PATCH,DELETE,requestdata andresponsestatuscodes

4. SourceCode/ImplementationSteps

```
importrequests

url =

"https://jsonplaceholder.typicode.com/posts"re

sponse = requests.get(url + "/1")
print("GET          Status:",
response.status_code) print("GET Data:",
response.json())

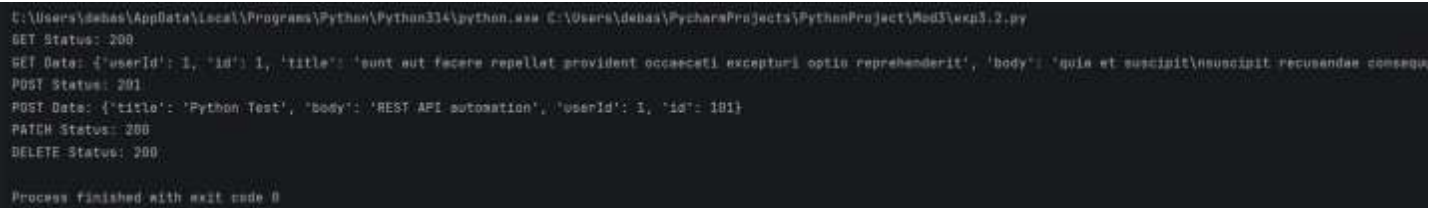
payload={
    "title":"PythonTest",
    "body": "REST API automation",
    "userId": 1
}

response      =      requests.post(url,
json=payload)  print("POST  Status:",
response.status_code)      print("POST
Data:", response.json())

response = requests.patch(url+ "/1", json={"title": "Updated Title"})
```

5.Output

5.1 TERMINALOUTPUT



```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\Mod3\exp3.2.py
GET Status: 200
GET Data: {'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur facere ducimus Lorem ipsum dolor sit amet consectetur adipiscing elit mattis dapibus ut mauris eget Lorem tempor metus faucibus et alia aliquam et enim enim nisi ut aliquam nunc sed justo augue euismod eros gravida risus varius blandit tempus porttitor rutrum amet consectetur adipiscing elit mattis dapibus ut mauris eget Lorem tempor metus faucibus et alia aliquam et enim enim nisi ut aliquam nunc sed justo augue euismod eros gravida risus varius blandit tempus porttitor rutrum amet'}
POST Status: 201
POST Data: {'title': 'Python Test', 'body': 'REST API automation', 'userId': 1, 'id': 101}
PATCH Status: 200
DELETE Status: 200

Process finished with exit code 0
```

Fig3.2.1:Terminaloutputshowingtheexecutionresult.

5.2 API RESPONSE

The REST API successfully processed GET, POST, PATCH, and DELETE requests and returned the corresponding HTTP status codes.

6.Result,Observation&Conclusion

Result:

The Python program was executed successfully, and GET, POST, PATCH, and DELETE requests were performed on the REST API.

Observation:

The GET request returned existing data with status code **200**. The POST request created a new record with status code **201**. The PATCH and DELETE requests were also completed successfully with status code **200**.

Conclusion:

This experiment demonstrated how Python's Requests library can be used to perform different HTTP operations. The terminal output confirmed that the API requests were sent successfully and the expected response data and status codes were received.

Experiment3.3:APIAutomation

1. ProblemStatement

Writing every APIrequest separately can make an automation script repetitive. The task is to use a common base URL, reusable functions and a session for API automation.

2. Objective

To create a simple reusable Python API automation program using functions and a RequestsSession.

3. Tools,Software&ConceptsUsed

- Python3.x
- PyCharm
- Requestslibrary
- RESTAPI
- Pythonfunctions
- RequestsSession

Concepts:globalconfiguration,reusablefunctions,sessionmanagement,APImethodsandJSONdata

4. SourceCode/ImplementationSteps

```
import requests

BASE_URL =
"https://jsonplaceholder.typicode.com" session =
requests.Session()

def get_post(post_id):
    response = session.get(f"{BASE_URL}/posts/{post_id}")
    print("Status:", response.status_code)
    print("Title:", response.json()["title"])

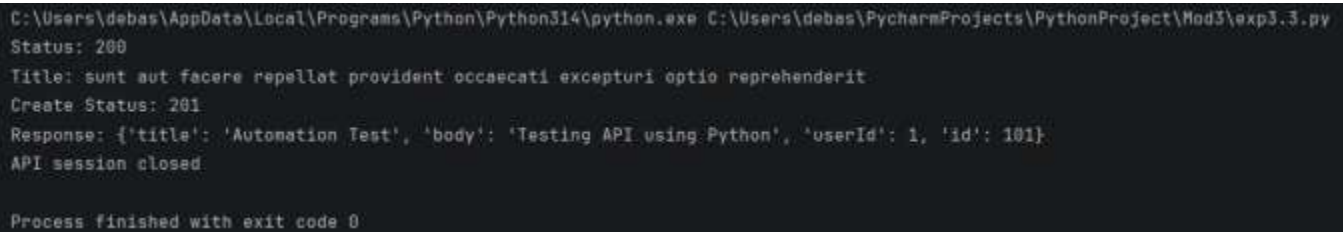
def create_post():
    payload = {
        "title": "AutomationTest",
        "body": "Testing API using Python",
        "userId": 1
    }

    response = session.post(f"{BASE_URL}/posts", json=payload)
    print("Create Status:", response.status_code)
    print("Response:", response.json())

get_post(1)
create_post()
```

5.Output

5.1 TERMINALOUTPUT



```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\Mod3\exp3.3.py
Status: 200
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Create Status: 201
Response: {'title': 'Automation Test', 'body': 'Testing API using Python', 'userId': 1, 'id': 101}
API session closed

Process finished with exit code 0
```

Fig3.3.1:Terminaloutputshowingtheexecutionresult.

5.2 API RESPONSE

TheAPIsuccessfullyreturnedtherequestedpostandcreatedanewrecordusingthePOSTrequest.

6.Result,Observation&Conclusion

Result:

The PythonAPI automationprogramwas executedsuccessfully. The GET request retrieved the required post, and the POST request created a new record successfully.

Observation:

The GET requestreturnedstatuscode **200** anddisplayedthe post title. The POST request returned status code **201** and displayed the newly created record. The API session was then closed properly.

Conclusion:

This experiment demonstrated howreusable Python functions and a Requests Session can be used for API automation. The terminal output confirmed that the API requests were processed successfully and the expected response data was received.

Experiment3.4:HandlingHTTPResponses

1. ProblemStatement

REST API tests need to check status codes and response content and also handle request errors. The task is to read the response object and use exception handling.

2. Objective

To understand response objects, status codes, headers, JSON content and exception handling in Python API automation.

3. Tools,Software&ConceptsUsed

- Python3.x
- PyCharm
- Requests library
- HTTP status codes
- Exception handling

Concepts: response object, status codes, response headers, response text/JSON, timeout and exceptions

4. SourceCode/ImplementationSteps

```
import requests

url =

"https://jsonplaceholder.typicode.com/posts/1"

try:
    response = requests.get(url, timeout=10)

    print("StatusCode:", response.status_code)
    print("ContentType:", response.headers.get("Content-Type"))

    if response.ok:
        data = response.json()
        print("Response ID:", data["id"])
        print("ResponseText:", data["body"])
    )
else:
    print("Request was not successful")
```

5.Output

5.1 TERMINALOUTPUT

```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\Mod3\exp3.4.py
Status Code: 200
Content Type: application/json; charset=utf-8
Response ID: 1
Response Text: quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto

Process finished with exit code 0
```

Fig3.4.1:Terminaloutputshowingtheexecutionresult.

5.2 API RESPONSE

The API returned a successful JSON response with status code 200, and the required response fields were displayed

6.Result,Observation&Conclusion

Result:

The Python program was executed successfully and received a response from the REST API with status code 200.

Observation:

The response content type was identified as JSON. The program successfully accessed the response ID and response text from the returned JSON data.

Conclusion:

This experiment demonstrated how HTTP responses can be handled and checked using Python. The terminal output confirmed that the request was successful and that the required response information was received and processed correctly.

Experiment3.5:BDD

1. ProblemStatement

Behaviour Driven Development allows a test scenario to be written in readable Given, When and Then steps. The task is to connect a Behave feature file with Python code that calls a REST API.

2. Objective

To understand the basic structure of a Behave BDDAPItest and connect readable Gherkin steps with Python step definitions.

3. Tools,Software&ConceptsUsed

- Python3.x
- PyCharm
- Behave
- Requestslibrary
- Gherkinsyntax
- REST API

Concepts:BDD,featurefile,scenario,Given/When/Then,stepdefinitionsandAPIvalidation

4. SourceCode/ImplementationSteps

```
#features/api_test.feature
```

```
Feature:REST APItesting
```

```
Scenario: Get a post from REST API
```

```
Given the REST API is  
availableWhen I send a GET  
request for post 1  
Thenthe response status should be 200
```

```
#features/steps/api_steps.py
```

```
importrequests  
frombehaveimportgiven,when,then
```

```
@given("the REST API is available")  
def step_api_available(context):  
    context.url="https://jsonplaceholder.typicode.com"
```

```
@when("I send a GET request for post  
1") def step_send_request(context):  
    context.response=requests.get(context.url+"/posts/1")
```

```
@then("the response status should be  
200") def step_check_status(context):
```

5.Output

5.1 TERMINALOUTPUT

A terminal window showing the execution of a Behave test. The prompt is PS C:\Users\debas\PycharmProjects\PythonProject>. The user enters cd Mod1, then cd exp5.5, and finally behaves. The output shows the runner path, the feature file, the scenario text, and the results: 1 feature passed, 0 failed, 0 skipped; 1 scenario passed, 0 failed, 0 skipped; 0 steps passed, 0 failed, 0 skipped. Total time is 0.008s.

```
PS C:\Users\debas\PycharmProjects\PythonProject> cd Mod1
PS C:\Users\debas\PycharmProjects\PythonProject\Mod1> cd exp5.5\
PS C:\Users\debas\PycharmProjects\PythonProject\Mod1\exp5.5> behave
USING RUNNER: behave.runner:Runner
Feature: REST API testing # features/api_test.feature:1

  Scenario: Get a post from REST API Given the REST API is available When I send a GET request for post 1 Then the response status should be 200 # features/api_test.feature:1

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
0 steps passed, 0 failed, 0 skipped
Total time 0.008s
```

Fig3.5.1:Terminaloutputshowingtheexecutionresult.

5.2 BDDOUTPUT

The Behavescenariowas executedsuccessfully. The GET request returned status code 200 and all three BDD steps passed.

6.Result,Observation&Conclusion

Result:

The BehaveBDDtestwasexecutedsuccessfully,andtheRESTAPIscenariopassedwithoutanyerrors.

Observation:

The Given step confirmed that the RESTAPIwas available. The When step sent a GETrequest for post 1, and the Then step verified that the response status code was 200. All three steps passed successfully.

Conclusion:

This experiment demonstrated the use of BDD with Python and Behave for REST API testing. The feature file described the test scenario using Given, When and Then statements, while Python step definitionsperformedtheactual API operations.Theterminaloutput confirmedthat the feature, scenario and all three steps were successful.